Assignment 1 Reinforcement Learning

MLDS 490 / IEMS 469 | October 25, 2026

Submission information	Value
GitHub username	jingliangguo2027
Repository	mlds-assignment-1
Repository URL	https://github.com/jingliangguo2027/mlds-assignment-1

Code and results: github.com/jingliangguo2027/mlds-assignment-1. The repository is private. The instructor must have access before submission.

1. Summary

I solved the shuttle problem with finite-horizon dynamic programming, value iteration, and policy iteration. I also used basic REINFORCE to train CartPole and Pong, both with and without a baseline. All results below come from saved runs. Failed or weak results are kept.

Game	Baseline	Training games	Test games	Mean score	Std. dev.
CartPole	none	2000	500	123.094	36.230
CartPole	ema	2000	500	369.722	65.631
Pong	none	5000	500	-18.374	2.743
Pong	ema	5000	500	-18.786	2.709

The standard deviation uses ddof=0. Test scores are sums of rewards, without discounting. The training objective uses discounted rewards. Each test uses a frozen stochastic policy for 500 full episodes.

Main findings

The shuttle methods agree on all 201 states. The infinite-horizon policy sends a shuttle when at least 13 people are waiting. The EMA baseline improves CartPole in this run. It improves Pong earlier in training, but the final Pong test score is slightly lower. Neither final Pong policy wins a test game.

How to read this report

Pages 2-4 cover the shuttle model, plots, and checks. Pages 5-10 explain policy gradient and show the required game plots. Pages 11-12 separate correctness checks from tuning experiments. Pages 13-14 give run commands and the three-month validation plan.

2. Shuttle model and algorithms

State s is the number of people waiting just before a decision: s = 0, ..., 200. Action a = 0 means wait; a = 1 means send one shuttle. The shuttle capacity is 15. Arrivals are independent and uniformly distributed over 1, 2, 3, 4, and 5.

```
remaining(s,a) = max(s - 15*a, 0)
cost(s,a) = 100*a + 2*remaining(s,a)
next_state = min(200, remaining(s,a) + arrivals)
gamma = 0.95
```

Order and assumptions

The agent observes the queue, chooses an action, and pays the dispatch and waiting costs. The waiting cost counts only people left after dispatch. New people arrive before the next decision. Arrivals above 200 are excluded from the queue; no extra penalty is charged. This cap is a modeling assumption, not an extra service mechanism. Sending an empty shuttle is allowed and costs 100. The terminal cost at time T is zero.

Bellman update

```
Q(s,a) = cost(s,a)
         + gamma * mean(V[next_state] for arrivals in 1..5)
V(s) = min(Q(s,0), Q(s,1))
```

The expectation averages all five next-state values. It does not replace random arrivals with their mean of 3: the value function is not generally linear.

Method	What the code does	Stopping rule
Finite horizon, T=500	Set V[500,s]=0. For t=499 down to 0, compare both actions at every state. Reuse the next row instead of expanding a full tree.	Finish all 500 rows. Plot V[0,s].
Value iteration	Start all values at zero. Each sweep reads the old value array and minimizes over both actions.	Maximum absolute change over all states < 1e-6.
Policy iteration	Start with wait everywhere. Evaluate the fixed policy, then improve every state using that same value array.	Evaluation tolerance 1e-8; stop when no action changes.

On exact ties, finite-horizon and value iteration choose wait. Policy iteration keeps the old action. The finite-horizon result is exact for the stated discrete model up to floating-point error. Infinite-horizon results are numerical approximations controlled by their tolerances.

Files: main.py; value_iteration.py; policy_iteration/main.py.

3. Shuttle value functions

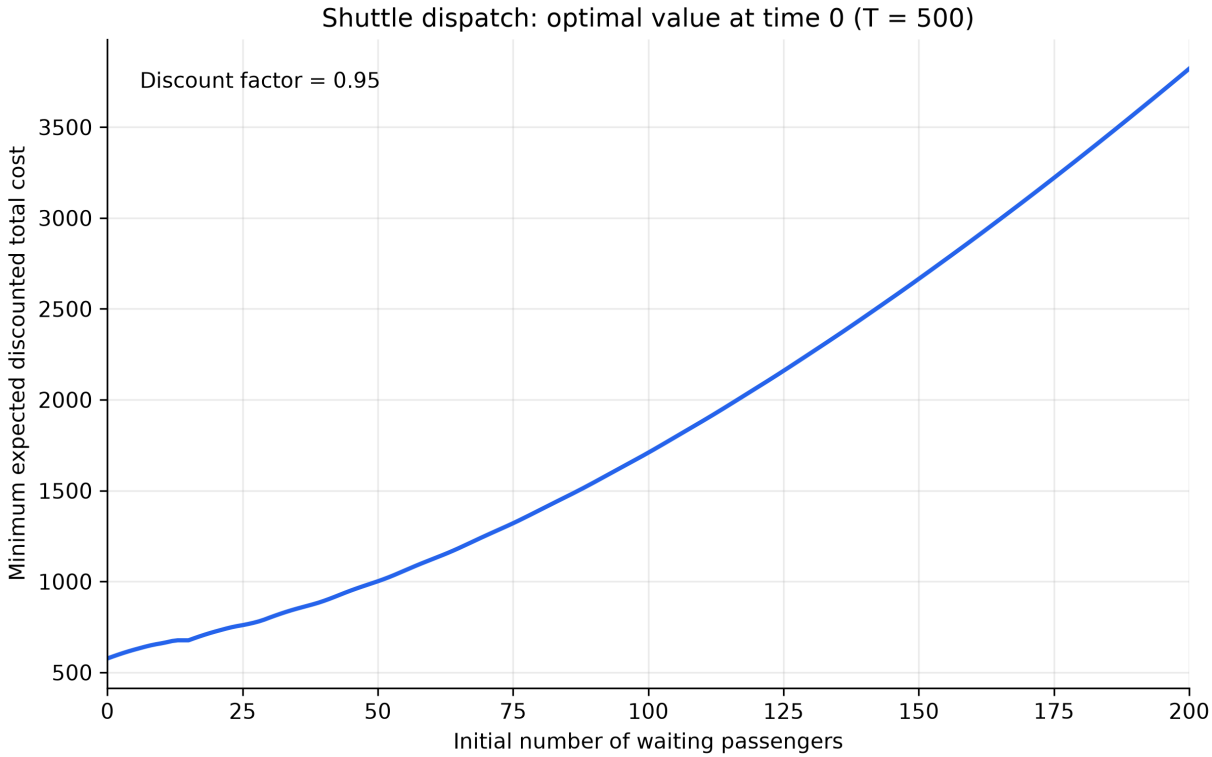


Figure 1. Minimum expected discounted cost at time 0, T=500. The value rises as more passengers need service.

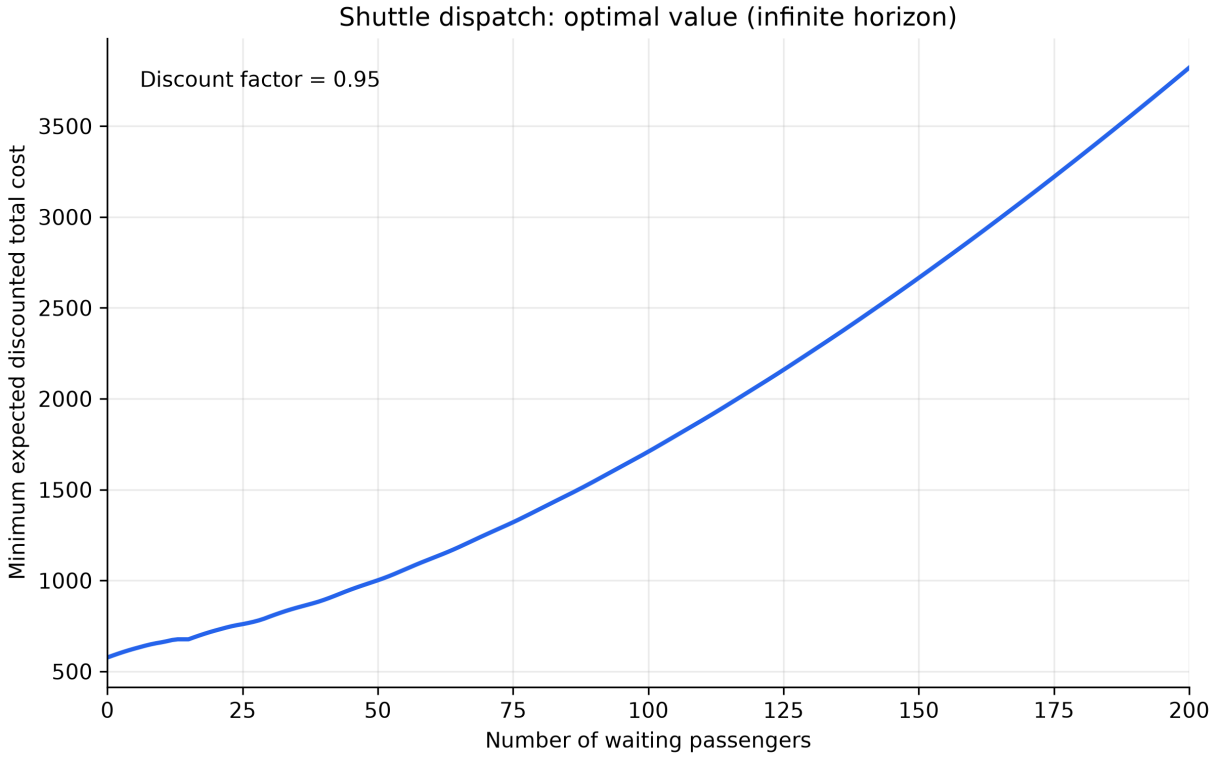


Figure 2. Infinite-horizon value iteration, gamma=0.95. The shape is close to Figure 1 because distant costs have very small discount weights.

Value iteration stopped after 340 sweeps. The maximum difference between finite T=500 and value iteration was 1.81e-5 over all 201 states. Costs are not simulation averages; the update explicitly averages the five possible arrivals.

4. Shuttle policy and validation

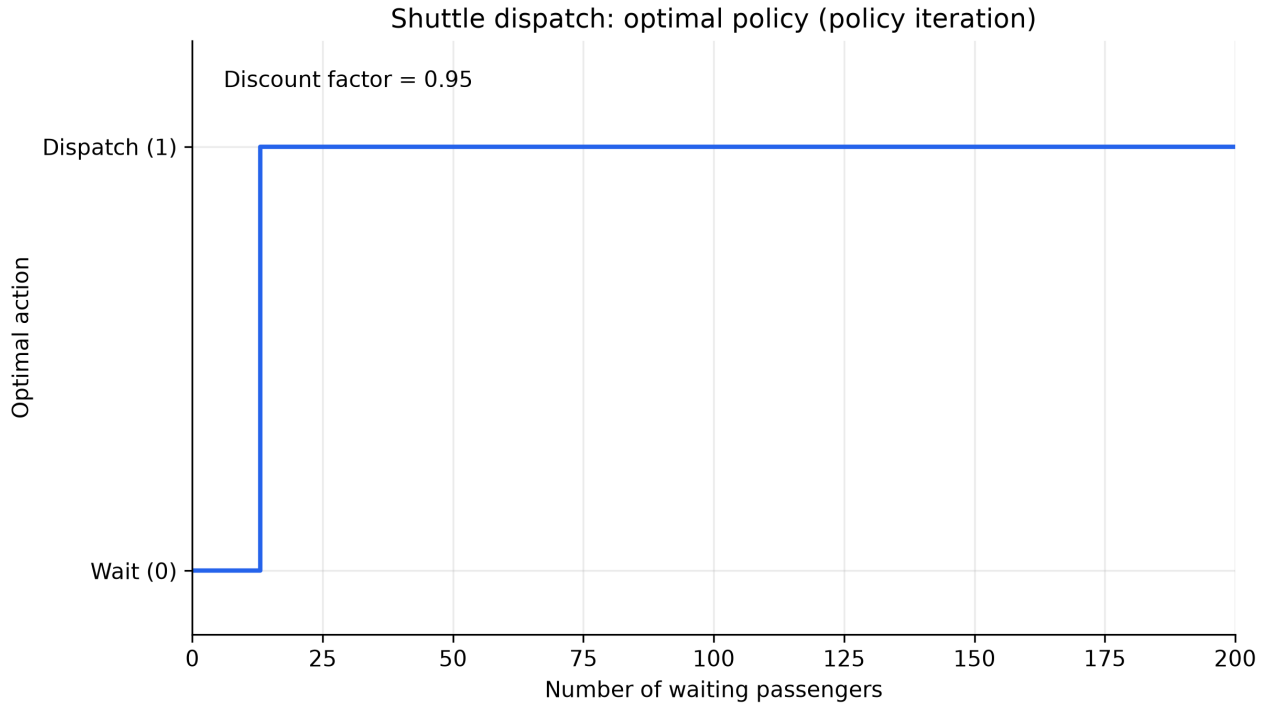


Figure 3. Infinite-horizon optimal action: wait below 13 passengers and dispatch at 13 or more. This result depends on the stated costs, capacity, cap, and discount.

Tests with known or independently checked answers

Test	Expected answer and reason	Observed result
Set gamma=0	Only this step matters. Sending costs at least 70 more than waiting, so every state should wait and V(s)=2s.	201/201 actions correct; maximum value error 0.
Empty / partial / full shuttle	At s=0, sending costs 100. At s=15, sending leaves 0 and costs 100. At s=16, it leaves 1 and costs 102.	All 7 hand-calculated cost checks passed.
Queue cap	From remaining 198, next states are 199,200,200,200,200. Probability at 200 must be 4/5.	All 3 transition-expectation checks matched.
Always wait at the cap	V(200)=400/(1-0.95)=8000. Neighbor states 198/199 also have analytic values.	Maximum error 1.89e-7; allowed error 2e-7.
Compare three algorithms	Long finite horizon and two infinite-horizon methods should be close under this discount.	201/201 actions agree for every pair; maximum value difference 1.81e-5.

Policy iteration stopped after 6 policy-evaluation/improvement rounds. Matching algorithms alone could hide a shared bug, so the zero-discount and analytic boundary tests provide separate checks.

Evidence: policy_iteration/validation_gamma0.json, validation_boundaries.json, validation_comparison.json. Run: python policy_iteration/validate.py --experiment all.

5. Policy gradient and baseline

I use one hidden ReLU layer and two output logits. A categorical distribution turns logits into action probabilities. During both training and testing, actions are sampled from these probabilities. I use Adam and update the policy once after each episode.

Setting	CartPole-v1	ALE/Pong-v5
Input / actions	4 state values; left or right	6400 frame-difference pixels; actions 2/3 only
Network	4 -> 32 -> 2	6400 -> 128 -> 2
Discount / learning rate	0.95 / 0.01	0.99 / 0.001
Training episodes	2000 per baseline setting	5000 per baseline setting
Initialization	PyTorch default	He-normal hidden weights; zero biases; orthogonal output weights, gain 0.01
Device	Local CPU	Deepdish 4, one RTX A6000, visible GPU 0

Pong images are cropped to rows 35:195, downsampled to 80x80, stripped of two background colors, and made binary. I subtract the previous processed frame to show motion. At reset the previous frame is zero. I keep v5 defaults, including frameskip 4 and sticky-action probability 0.25. There is no reward shaping or reset of returns after individual Pong points.

```
G[T] = 0
G[t] = r[t] + gamma * G[t+1]       # work backward
L = -sum_t gamma^t * (G[t] - b) * log pi(a[t] | s[t])
```

The outer gamma^t is retained in the final experiments. For a long episode, gradients are accumulated in chunks of 256 states, then Adam takes one step. No clipping, return normalization, critic network, or entropy bonus is used.

Baseline: one historical moving average

```
b = 0                              # at the start of training
# For each episode:
update_policy_using(G - b)         # use the previous b
reference = sum(gamma^t * G[t]) / sum(gamma^t)
b = 0.9*b + 0.1*reference          # used in the next episode
```

Without a baseline, b stays zero. With EMA, b is a scalar and is not learned by backpropagation. It is updated after the policy step, so it does not use the current action when serving as the current baseline. It can reduce gradient variance without guaranteeing a higher final score.

Episodes end on either termination or time-limit truncation; no value is bootstrapped beyond that endpoint. Test runs freeze both the policy and the baseline. The reward at an individual step is supplied by the environment, not chosen by hand.

6. CartPole without baseline

2000 training episodes; gamma=0.95; Adam lr=0.01.

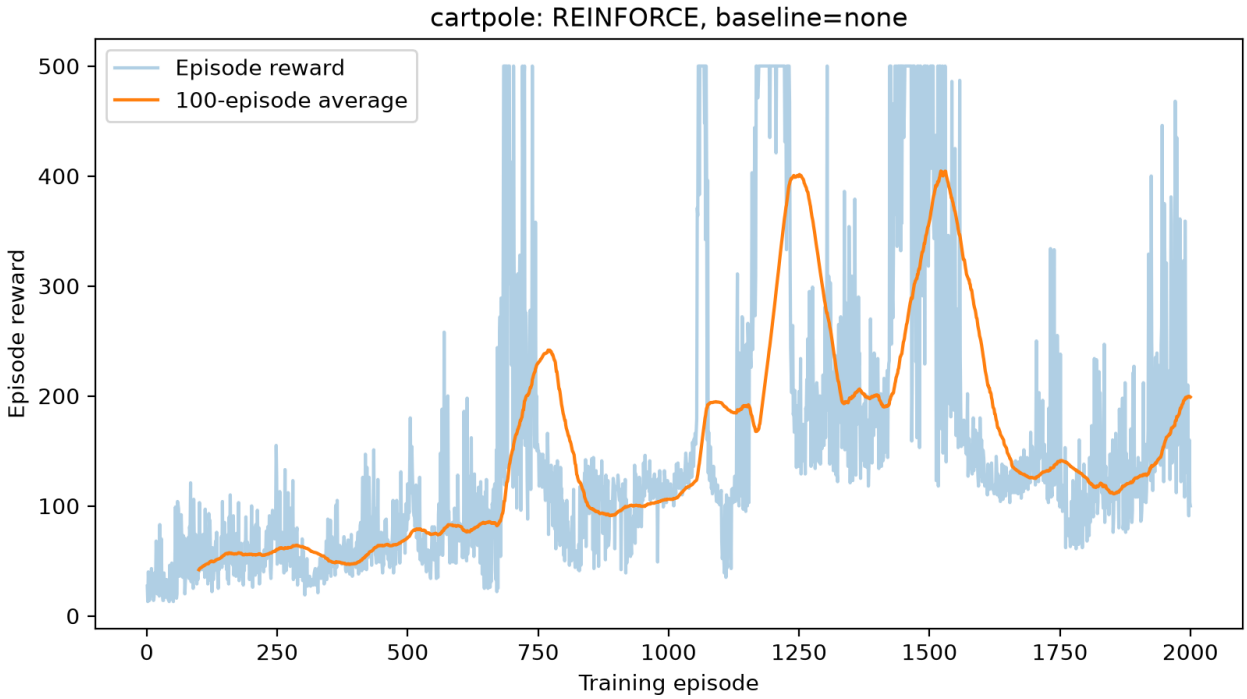


Figure 4. Training: each episode score and the trailing 100-episode moving average. The first average is at episode 100.

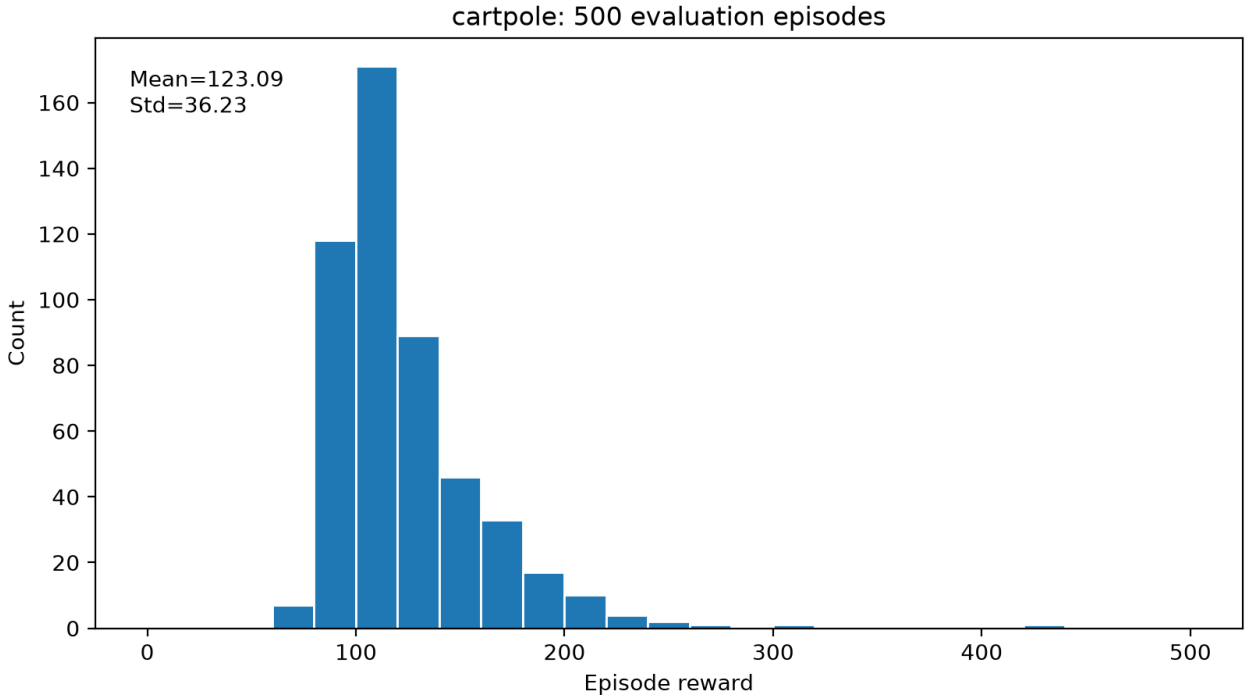


Figure 5. Final test:500 frozen-policy episodes. The histogram uses raw episode reward; standard deviation uses ddof=0.

The final policy has a mean score of 123.094. Training improves, but the policy does not reliably keep the pole up for 500 steps.

7. CartPole with EMA baseline

Same initial seed, network, learning rate, and 2000-episode budget.

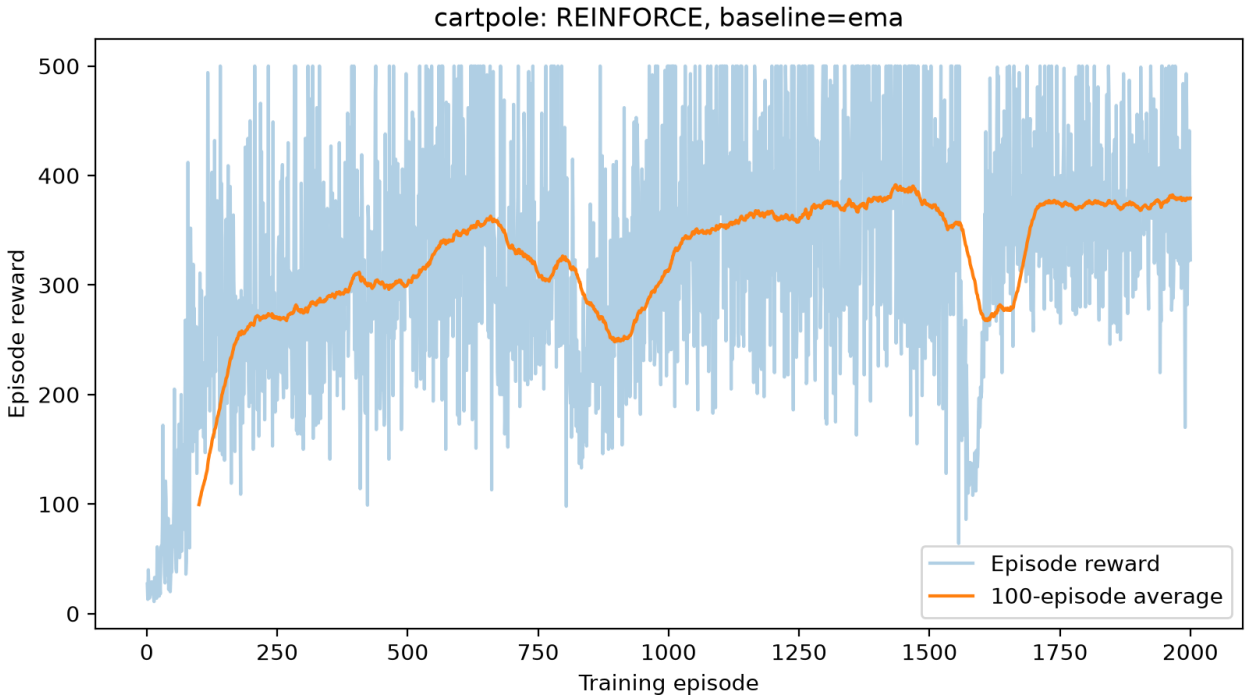


Figure 6. Training: each episode score and the trailing 100-episode moving average. The first average is at episode 100.

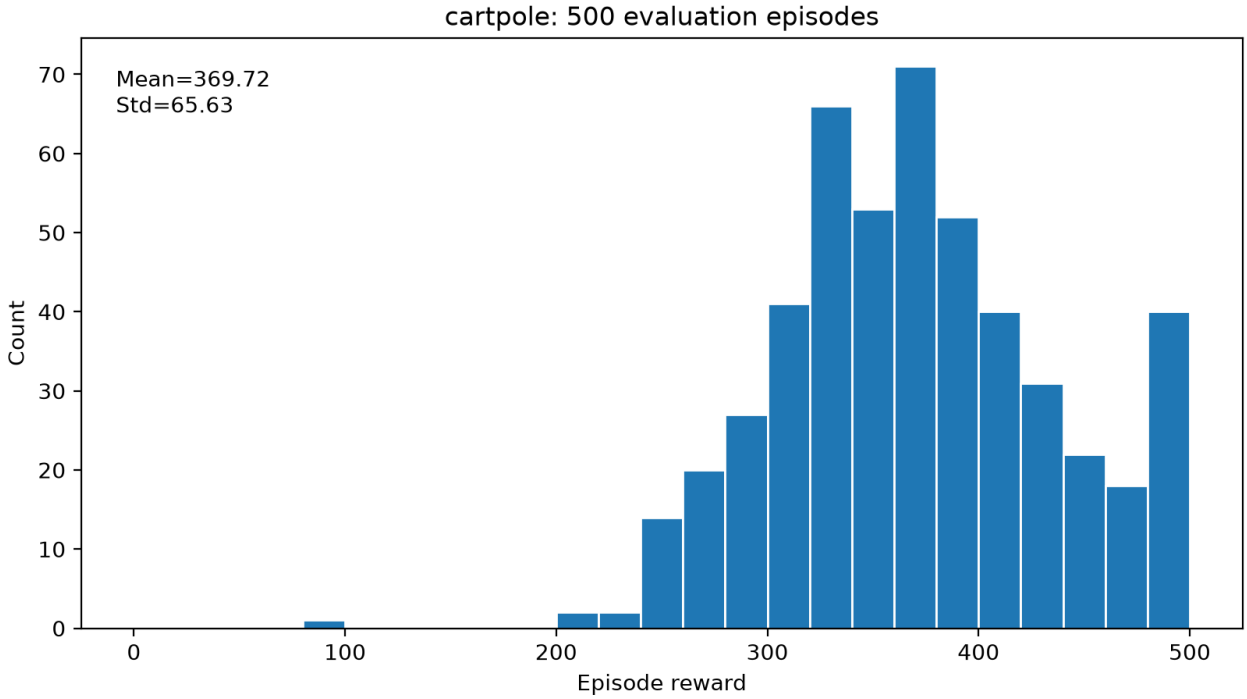


Figure 7. Final test:500 frozen-policy episodes. The histogram uses raw episode reward; standard deviation uses ddof=0.

The final mean is 369.722, compared with 123.094 without baseline. The gain is 246.628 points in this pair of runs; this is not a multi-seed result.

8. Pong without baseline

5000 training episodes; gamma=0.99; Adam lr=0.001; 128 hidden units.

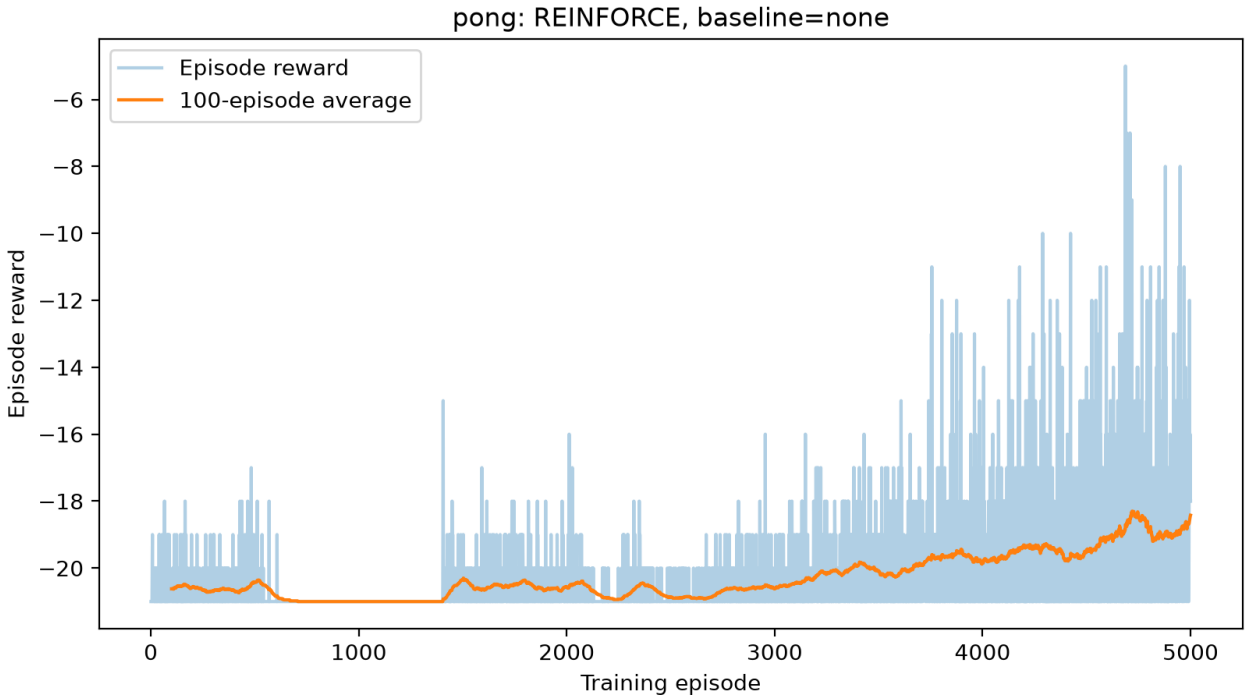


Figure 8. Training: each episode score and the trailing 100-episode moving average. The first average is at episode 100.

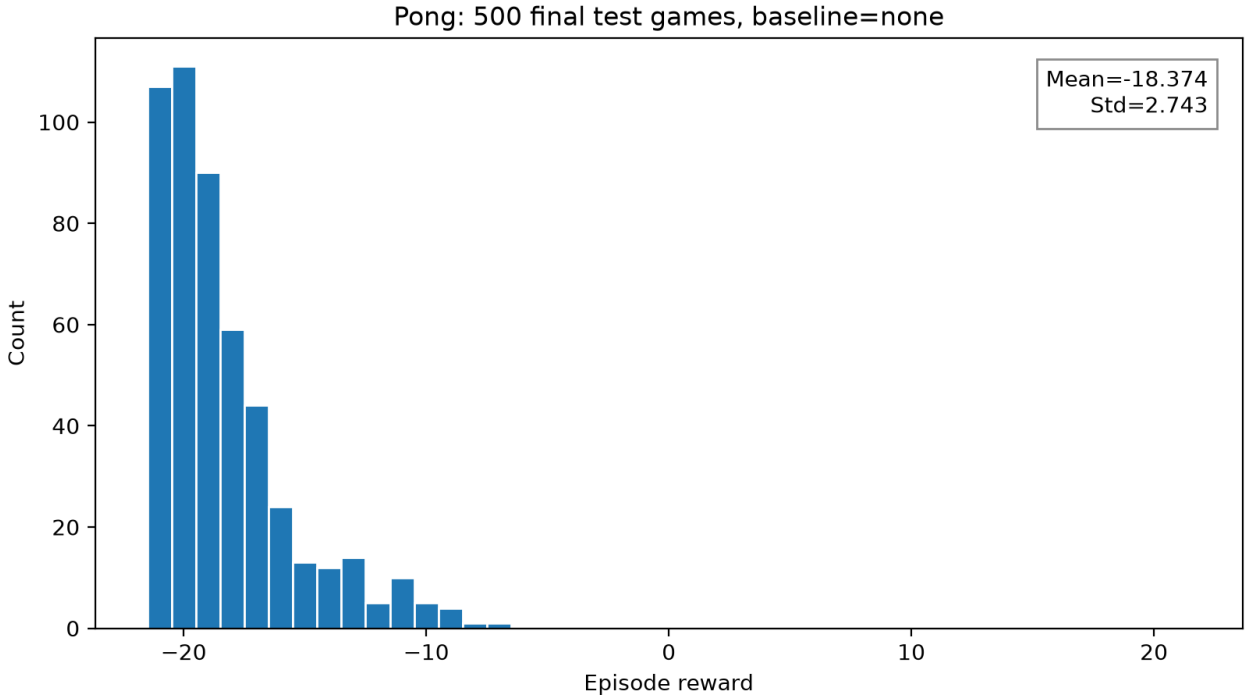


Figure 9. Final test:500 frozen-policy episodes. The histogram uses raw episode reward; standard deviation uses ddof=0.

The model improves late in training, but all 500 final test games are losses. Mean=-18.374; standard deviation=2.743.

9. Pong with EMA baseline

Same starting weights and 5000-episode budget; only the EMA baseline is added.

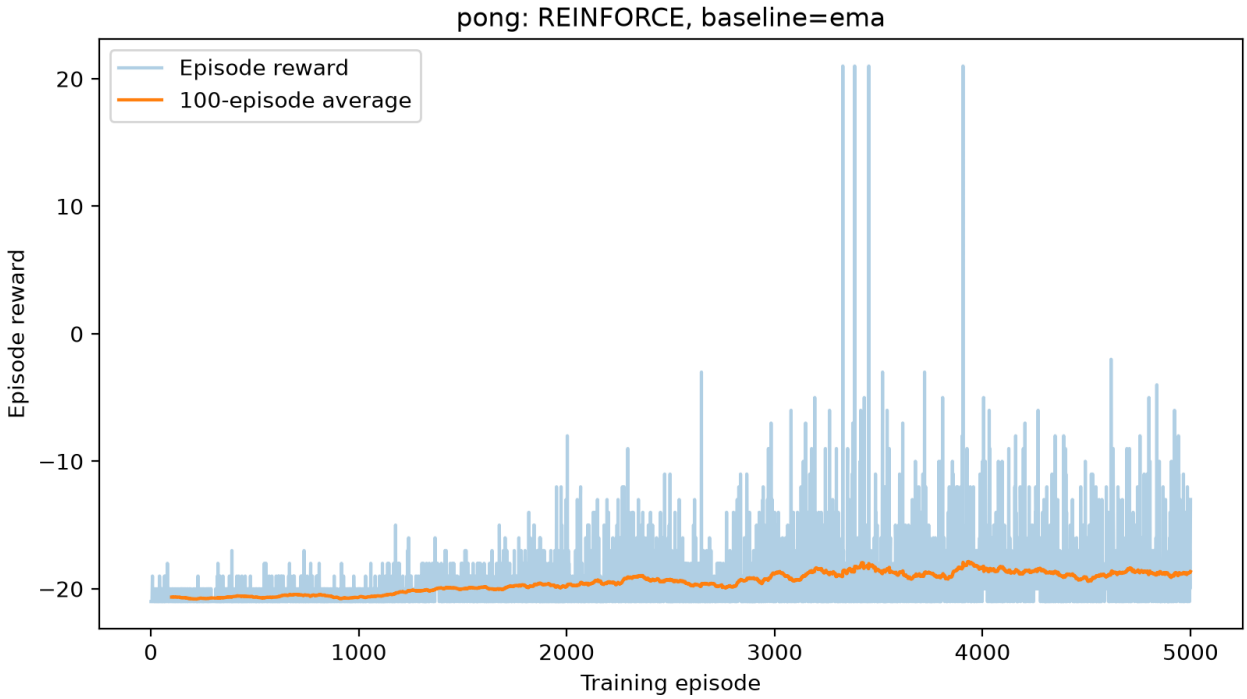


Figure 10. Training: each episode score and the trailing 100-episode moving average. The first average is at episode 100.

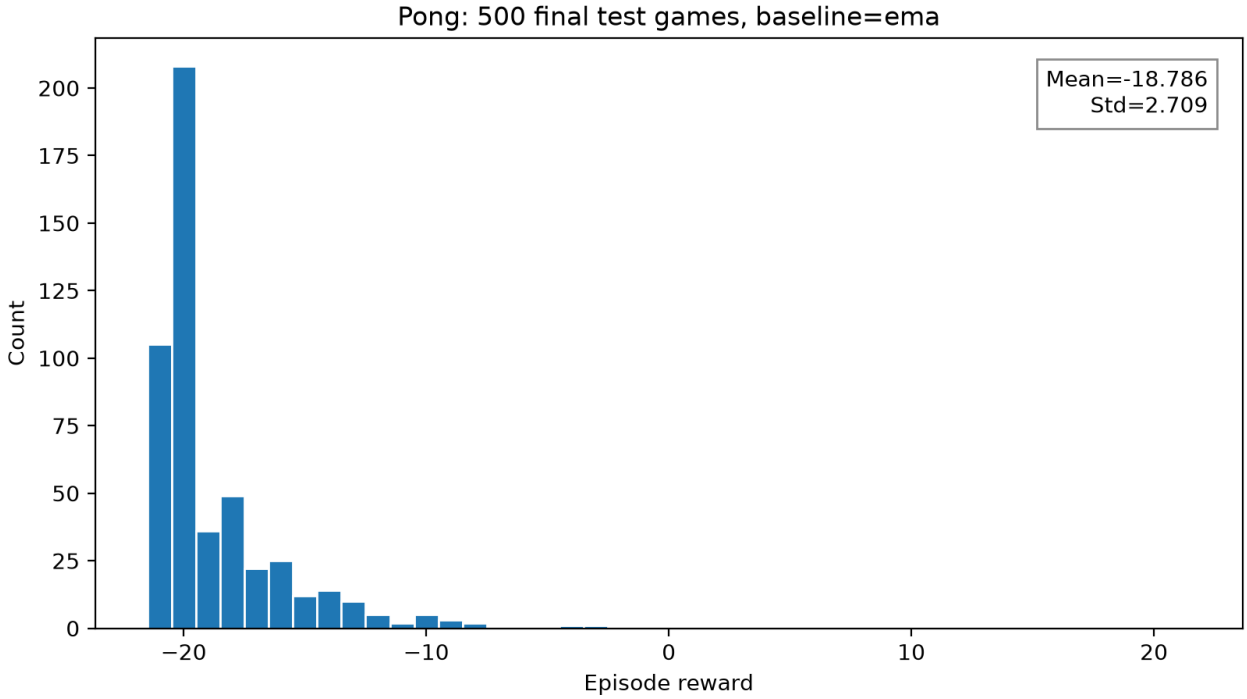


Figure 11. Final test:500 frozen-policy episodes. The histogram uses raw episode reward; standard deviation uses ddof=0.

The baseline helps earlier in training, but all 500 final test games are losses. Mean=-18.786; standard deviation=2.709.

10. Comparing the two baselines

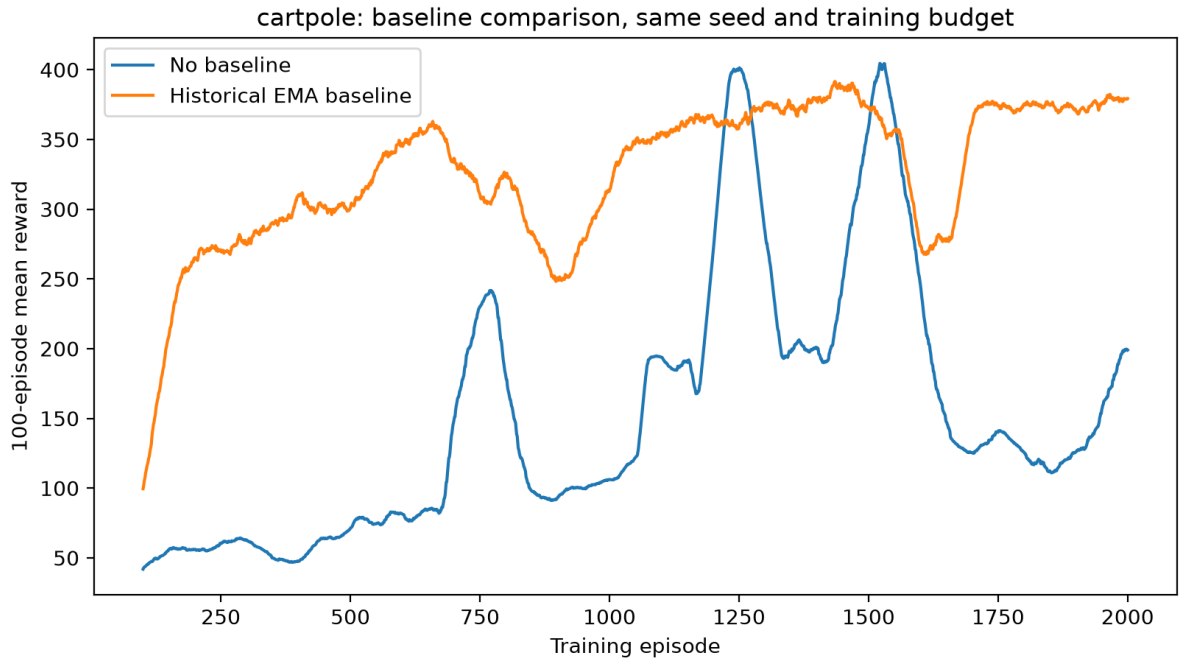


Figure 12. CartPole: EMA reaches a higher training level in this matched pair.

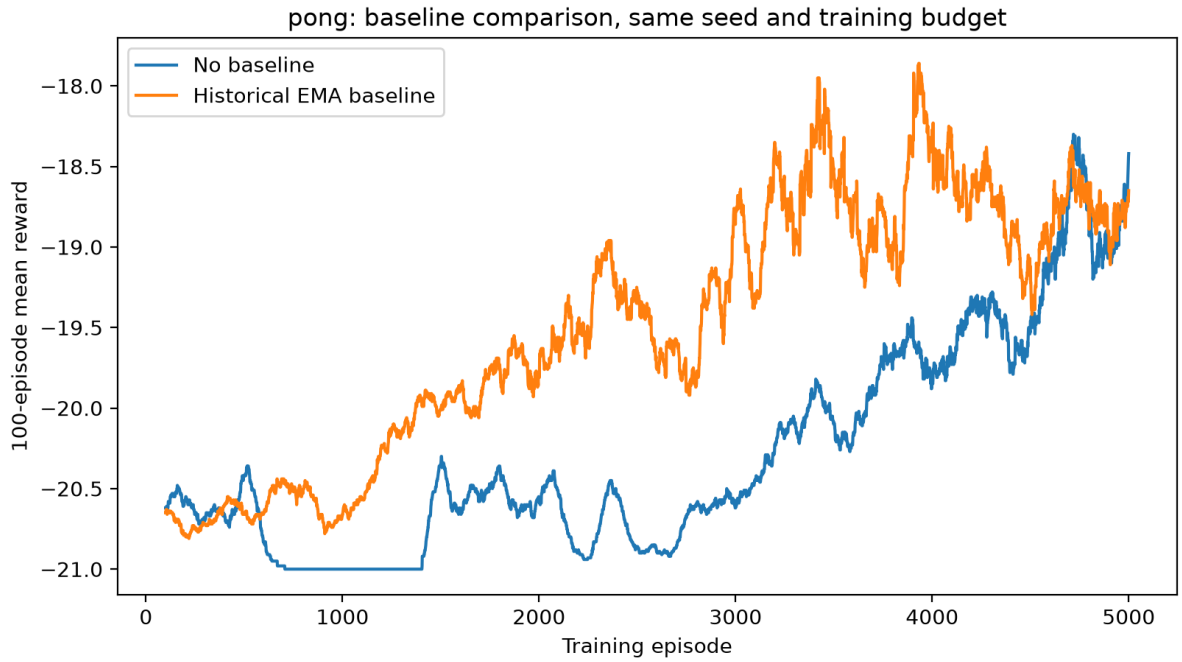


Figure 13. Pong: EMA improves earlier. The no-baseline model catches up toward the end.

The Pong final mean difference (EMA minus no baseline) is -0.412. A paired bootstrap over the same 500 test seeds gives a95% interval of[-0.706,-0.130]. Thus no baseline is slightly better for these two fixed models on this test. This interval does not include variation from retraining with other seeds.

Pong mean discounted test return is-0.204 with EMA and-0.264 without it. EMA is better on this discounted metric but worse on whole-game score. The two measurements answer different questions. Final tests use seeds 1000000-1000499; the 50-game tuning checks use 200000-200049.

11. Correctness checks and edge cases

A correctness test asks whether the algorithm gives the answer predicted by its equations. A tuning test asks whether a setting gives better results. Good tuning scores alone do not prove that gradients or transitions are correct.

Check	Prediction / purpose	Saved evidence
Exact two-step gradient	Enumerate 4 action sequences. Compare the expected REINFORCE gradient with the direct gradient of J=p(0)+0.95*2*p(1). Repeat for b=0 and b=0.8.	Maximum errors 8.94e-8 and 1.49e-8, both below 1e-6. Supports the sign, return formula, outer discount, and action-independent baseline.
Chunked vs full gradient	Splitting an episode for memory should preserve its summed gradient.	Relative error 8.46e-8 on an 887-step Pong trajectory. A small gradient-descent step also lowered the fixed-trajectory loss.
Paired gradient variance	Use 20 separate CartPole pilot episodes to set b. Freeze the policy and compare both gradients on the same 40 new trajectories.	Sum of component sample variances:891.86 without b;151.81 with b. Ratio 0.1702. This checks variance directly, not score variance.
Fixed-seed Pong replay	Keep policy, environment seed, and action seed fixed. Actions and rewards should repeat.	Both full 946-step trajectories matched; score=-21. This is a reproducibility check, not proof of learning.
Resume and test isolation	A split run should match uninterrupted training. Old scores must not be reused after a model changes.	CartPole EMA histories, weights, baseline and tests matched exactly; deliberately stale scores were rejected.

Task-specific boundaries

Pong inputs contain only the intended foreground pixels; preprocessing leaves the original image unchanged. The code maps network action 0 to environment action 2, and 1 to 3. At every reset the previous frame is cleared. Both episode-ending flags are handled. Log probabilities are computed from logits for numerical stability, and non-finite losses are rejected. These implementation safeguards support, but do not replace, the algorithm checks above.

First 10 steps of the saved fixed-seed Pong check

Step	1	2	3	4	5	6	7	8	9	10
Action	3	3	3	3	2	2	3	3	2	2
Reward	0	0	0	0	0	0	0	0	0	0
P(a=1)	0.529	0.529	0.532	0.536	0.535	0.532	0.525	0.530	0.539	0.528

Early rewards are zero because no point has yet been scored. The trace shows valid stochastic actions, not a manually assigned return. Evidence: policy_gradient/validation_results.json, resume_validation.json, tuning_results/audit.json and validation_checks.json.

12. Tuning and weak Pong performance

All tuning observations below are separate from the final 500-game tests. I kept experiment names, seeds, model states, and decisions. I changed one main factor at a time where possible. These runs are useful evidence, but their repeated use of a small validation set limits model selection.

Experiment	Budget and observation	Interpretation
Learning rate	At 100 episodes, lr 0.01 had mean-20.90 and 72.2% saturated actions in the early-step probe. lr 0.0001 had mean-20.55. These used 20 validation games.	The suggested 0.01 was not a clearly good Pong setting. The final runs use 0.001, a documented choice, not a proven optimum.
Network width	At 300 episodes and 50 validation games:32=-20.42;64=-20.40;128=-20.32;256=-20.84, all with default initialization.	More hidden units did not automatically improve this short run. This does not settle the long-run comparison.
Initialization	128 with He hidden/small output initialization scored-20.70 at 300 episodes.	No short-run advantage. I retained it for a controlled longer-budget test.
Longer training	The same no-baseline model improved from-20.70 at 300 to-18.60 at 5000 on 50 tuning games.	300 episodes were not enough to show its later improvement.
EMA baseline	At 5000,50-game tuning means were-18.60 without and-18.42 with EMA; their score-difference interval crossed 0.	No clear tuning advantage. The independent final 500-game test favors the no-baseline model slightly.
Remove outer discount	A separate historical branch omitted gamma^t from the loss. At 600 episodes it scored-19.80 on 50 tuning games.	This changes the gradient estimator. The final runs restore the original formula; the branch is not relabeled as the final method.

Why can the score remain poor?

Pong rewards are sparse and games are long. With gamma=0.99, a reward 500 steps later is weighted by about 0.00657; at 1000 steps the weight is about 0.0000432. The objective puts much more weight on early events. Better discounted return may therefore fail to improve the full-game score.

Exploration also decreased. In the 50-game tuning probes, the share of first 256-step action probabilities above 99% rose to 40.5% without a baseline and 73.4% with EMA. This is a possible reason for limited improvement, not a proven causal explanation. The one-layer pixel policy and per-episode updates may also limit learning. Further experiments would be needed to separate these causes.

I do not claim that the final policies solve Pong. I also do not claim that EMA always reduces test-score variation or always improves learning. The direct variance test and the final performance comparison measure different things.

13. How to run the code

All paths below are relative to the hw1 folder in the repository. Keep the existing results as evidence and use new output folders for new runs. Install PyTorch for your machine; then install the small dependency files. The report itself needs reportlab.

```
python -m pip install numpy matplotlib
python -m pip install -r policy_gradient/requirements.txt
python -m pip install reportlab

# Shuttle answers and validation
python main.py
python value_iteration.py
python policy_iteration/main.py
python policy_iteration/validate.py --experiment all
```

Reproduce the game methods and training budgets

```
cd policy_gradient
# Run these commands once for b=none and once for b=ema.
# Replace BASELINE with none or ema.
python train.py --game cartpole --baseline BASELINE \
  --episodes 2000 --eval-episodes 500 --lr 0.01 \
  --device cpu --output results/new_cartpole_BASELINE

CUDA_VISIBLE_DEVICES=0 python train.py --game pong \
  --baseline BASELINE --episodes 5000 --eval-episodes 500 \
  --hidden 128 --initialization he_small_output --lr 0.001 \
  --device cuda --output results/new_pong_BASELINE
```

The saved Pong runs used tune_pong.py for training and evaluate_pong_final.py for the final tests. The no-baseline run resumed from 300 episodes with its Adam state intact; the EMA run started fresh. Both began with identical seed 0 weights. The commands above reproduce the method and budget, not a promise of bitwise equality across devices or library versions.

Run and inspect the validation experiments

```
# From policy_gradient/
python validate.py
python validate_tuning.py
python validate_loss_modes.py
python test_resume.py
python summarize.py --require-complete

# From hw1/: rebuild this PDF from saved JSON and figures
python build_report.py
```

validate.py checks the exact gradient, Pong replay/preprocessing, and paired CartPole gradient variance. validate_tuning.py checks the exact gradient and batch-gradient averaging. validate_loss_modes.py checks both historical loss forms. test_resume.py is an additional engineering check. The original GPU audit can be rerun with CUDA_VISIBLE_DEVICES=0 python tune_pong.py --audit --output smoke/recheck_audit; it uses 20 tuning-validation games, not final test seeds.

Environment record: CartPole used PyTorch 2.14.1 on CPU, Gymnasium 1.4.0, NumPy 2.5.4. Pong used PyTorch 2.14.1+cu 126, Gymnasium 1.4.0 and ALE 0.12.1 on one RTX A6000. Deepdish used the myenv conda environment. No multiple-GPU training was used.

14. If I had three more months

The goal would be to learn which conclusions are repeatable, why Pong remains weak, and where the code could fail. I would keep correctness tests separate from model selection and keep a new final test set untouched until the settings are fixed.

Period	Tests and experiments	What I would measure
Month 1: reliability	Repeat each game/baseline setting with at least 5 training seeds. Add short problems with exact values and exact gradients. For the shuttle, test deterministic arrivals, zero arrival rate, zero dispatch cost, changed capacity, and alternative queue-cap rules.	Mean and uncertainty across training seeds; exact-solution errors; Bellman residuals and policy agreement. Separate repeated-run variation from test-game variation.
Month 2: controlled comparisons	Use matched budgets to change one factor at a time: learning rate, batch size, hidden width, initialization, or baseline. Compare a constant baseline, historical EMA, and a simple state-dependent baseline. Measure gradient variance at several frozen checkpoints on shared trajectories.	Learning speed, final score, discounted return, gradient variance, action entropy, time and memory. Do not assume that a smaller gradient variance guarantees better scores.
Month 3: robustness and final evaluation	Test unusual starting states and near-terminal states. For diagnostics, compare deterministic settings with the normal stochastic game. Study very long games and the effect of discounting. Then freeze the selected method and use new final test seeds.	Performance failures, robustness to changes, confidence intervals, and whether improvements survive independent testing. Keep the required v5 setup for the main result.

Limits of the current evidence

The current results use one training seed. The 500-game tests describe these trained models, not every run of the algorithm. The tuning sets were reused, so small tuning differences can be misleading. A tiny analytic example strongly checks a gradient formula but cannot prove every part of a large program correct. Replay checks test reproducibility, not whether a policy is good.

The main code is basic: arrays and loops for the shuttle, and a two-layer network with one optimizer step per episode for policy gradient. The extra scripts save results, run checks, and draw figures. No new training or parameter changes were made while writing this report.

Sources and saved evidence

Task requirements: Assignment 1, sections 1-4, and MLDS Lab Week 1 setup instructions. No external performance target was added. The report uses the recorded data listed below; no test outcomes were invented.

```
policy_iteration/validation_*.json
policy_gradient/validation_results.json
policy_gradient/resume_validation.json
policy_gradient/tuning_results/audit.json
policy_gradient/tuning_results/validation_checks.json
policy_gradient/tuning_results/stage_decisions.json
policy_gradient/results/cartpole_{none,ema}/metrics.json
policy_gradient/results/pong_final_{none,ema}/metrics.json
policy_gradient/final_test_comparison.json
```

Final Pong tests are tied to the selected model fingerprints and checkpoint hashes in tuning_results/final_test_500.json and final_test_verification.json. Raw scores, figures, code, and this report are included in the repository; large model files and local environments are kept outside Git.